Mo Tu We Th Fr Sa Su

Memo No.
Date / /

Threads & Concurrency

* A thread is a basic unit of CPU utilization It consists of thread ID, program counter (PC), register set, and a stack.

- It shares with other threads belonging to the same process its code section, data section and other OS resources like files.

~~Examples of multith~~

~~A process Most processes~~

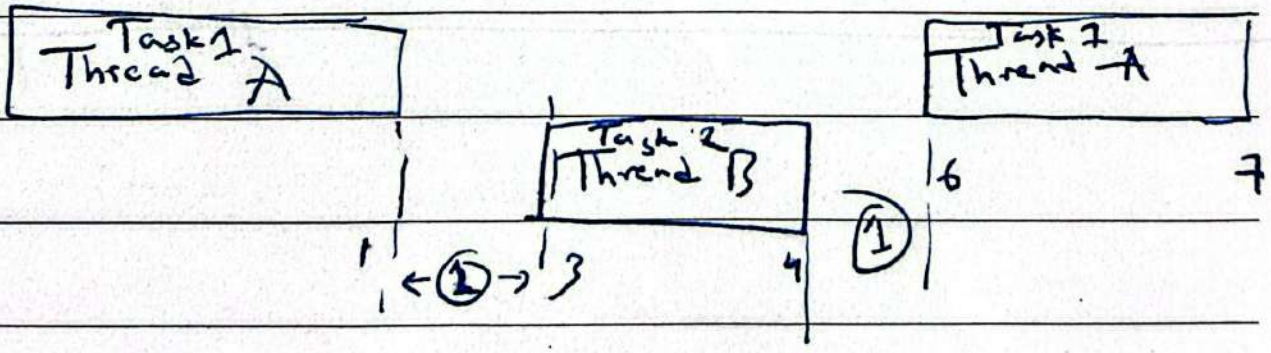
- A traditional process is single-threaded i.e it has a single thread of control.

- If a process is multithreaded, it can perform multiple tasks concurrently (using a single core CPU) or in parallel (using multicore/multiprocessor CPUs)

- On a single-core CPU, ~~the core cannot execute 2 threads simultaneously~~ multithreading may ~~does~~ not always make a program complete faster — in fact it can make it slower

- When you run a multithreaded process on a single-core CPU, the core cannot execute 2 threads simultaneously

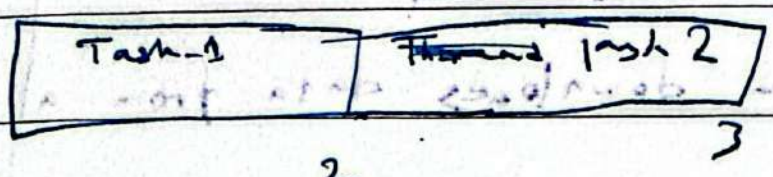
- The scheduler rapidly switches between threads - each gets a short time slice.



context switch overhead

Total CPU time = Time (Thread A) + Time (Thread B) + Context switch overhead

If you used Single thread



So why even use threads on a single core?

* Improves system responsiveness, ~~and resource utilization~~ resource sharing, economy, scalability.

Better Responsiveness

Example: A text editor

Thread 1: Handles keyboard input

Thread 2: Saves files in the background

App feels responsive because while Thread 2 waits for disk I/O, the scheduler gives CPU time to Thread 1

Date / /

Resource Sharing

- Processes can share resources only through techniques such as shared memory and message passing. Such techniques must be explicitely arranged by the programmer.
- Threads share the memory and the resources of the process to which it belongs to by default.
- The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.

Economy

~~Thread creation is less computationally expensive than~~

- Thread Creation consumes less time and memory than process creation.
- Context switching is ~~faster~~ between threads than processes.

Scalability

- In multi-core or multiprocessor systems, threads can run in parallel on different processing cores.

$$\text{Speed Up} = \frac{1}{(1-P) + \frac{P}{N}}$$

Sequential — Parallel

```
{
    int a = 1 + 2;
    int b = 3 * 3;
    int c = a + b;
}
```

Can be parallel

$\frac{1}{3}$ → Serial / Sequential

$\frac{2}{3}$ → Parallel

$$= \frac{1}{S + \frac{(1-S)}{N}}$$

If 1 core,

$$= \frac{1}{\frac{1}{3} + \frac{1 - \frac{1}{3}}{1}}$$

$$= \frac{1}{1}$$

If 2 cores

$$= \frac{1}{\frac{1}{3} + \frac{1 - \frac{1}{3}}{2}}$$

$$= 1.5$$

Mo | Tu | We | Th | Fr | Sa | Su

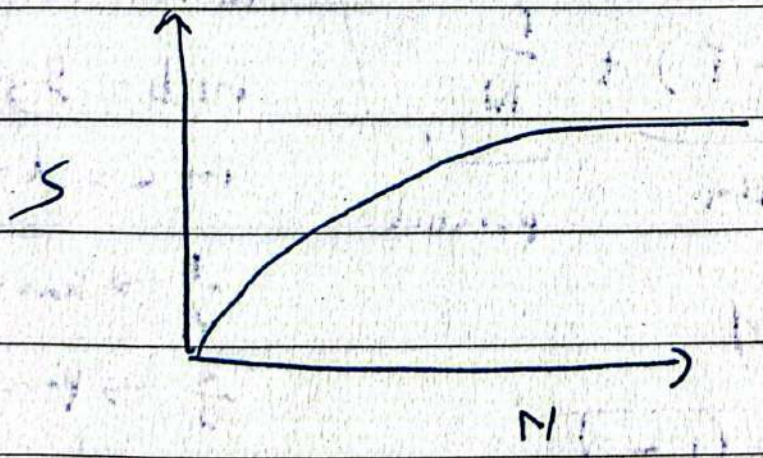
* In reality, ~~since there~~ the ~~extra cores won't~~ Sp
N cannot be more than 2 for this example.

$N = 10000$

$$Speedup = \frac{1}{\frac{1}{3} + \frac{1-\frac{1}{3}}{1000}}$$

$$\approx 2.999$$

$$Speedup_{max} \leq \frac{1}{S}$$



Types of Threads

1) User threads - Managed without kernel support

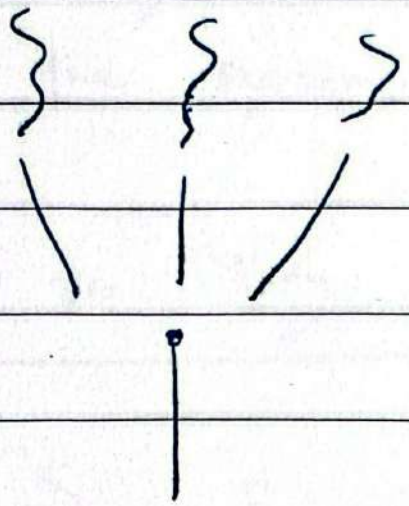
2) Kernel ~~Threads~~ Threads - Managed directly by the OS.

There must exists a relationship between user threads & kernel threads

3 ~~many~~ ways to establish this relationship!

1) Many-to-One Model
2) One-to-One Model
3) Many-to-Many Model

Many to One Model



(K) Kernel Thread

- Maps many user-level threads to one kernel thread
- Thread ~~Many~~ management ~~is done by the thread~~ such as context switch is done in the user space, so it is ~~effic~~ fast.
- The entire process will block if a thread makes a blocking system call
- Because only one thread can access the kernel at a time, multiple threads are

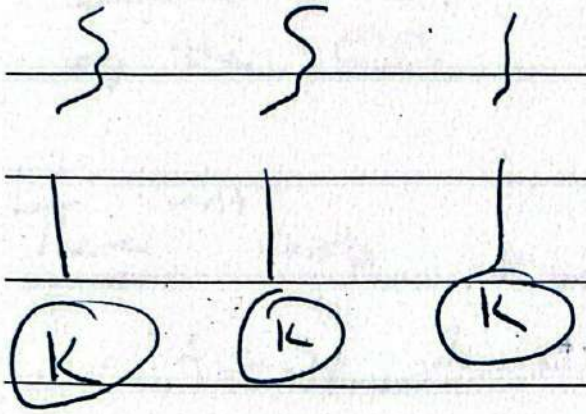
Mo Tu We Th Fr Sa Su

Memo No.

Date / /

unable to run in parallel on multiprocessors

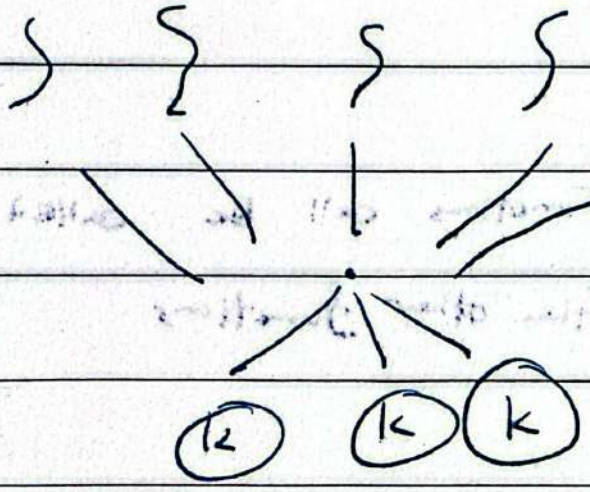
One to One Model



- Maps each user thread to a kernel thread.
- ~~Provides more concurrency than~~
- Allows another threads to run when while one is ~~blocked~~ making a blocking system call
- Also allows multiple threads to run in parallel on multiprocessors.

- Creating a user thread requires creating the corresponding kernel thread, which may burden the performance of the system.

Many to Many Model



- ~~Best of the~~
- Overcomes the limitations of the previous 2 models.
- E.g. User threads : T_1, T_2, T_3, T_4, T_5

  Kernel threads : K_1, K_2

  2 CPU cores